JAIN UNIVERSITY, BENGALURU

Department of Computer Science and Engineering

Mini-Project Report

22CSE544 Data Streaming and Analysis

WikiRadar: Real-Time Anomaly Detection on Live Wikipedia Edits

Wikimedia EventStreams, Kafka, Python, scikit-learn, PostgreSQL, Grafana

Submitted by

Likhith (23BTRCL257)

B.Tech CSE (AI/ML), Section AIML-D

Under the guidance of

Shahad P

October 2026

Contents

Abstract3
1. What Is Real-Time Streaming Analytics?3
2. What the Mini-Project Contains3
3. The Dataset: Live Wikipedia Edits4
4. Layers 1 and 2: Streaming the Data with Kafka5
5. Layer 3: The Stream Processor (Windowing)6
6. Layer 4: The Machine-Learning Model8
7. Layer 5: The Database (Serving Layer)9
8. Layer 6: The Live Dashboard10
9. Running the Whole Pipeline11
10. Demonstrating It Live11
11. Results12
12. Limitations and Future Work15
13. Conclusion15
References16
Appendix: Submission Checklist16

List of Figures

Figure 1: The six layers of WikiRadar4
Figure 2: Docker services running and healthy5
Figure 3: The Kafka topic wiki-edits5
Figure 4: Producer in live mode6
Figure 5: Processor output8
Figure 6: Training output9
Figure 7: Row counts and latest windows in PostgreSQL10
Figure 8: WikiRadar dashboard on live traffic11
Figure 9: Dashboard after a vandal_wave attack12
Figure 10: Edit war flagged12
Figure 11: Offline evaluation output13
Figure 12: Contamination sweep13
Figure 13: Test suite passing15

Abstract

WikiRadar watches every edit to English Wikipedia as it happens and flags 10-second periods that look unusual, such as a wave of anonymous vandalism, an edit war, or a bot flooding the site. Edits arrive from the Wikimedia EventStreams feed, pass through Apache Kafka, and are grouped into 10-second windows by the time each edit was made. Each window becomes 11 features, and an IsolationForest trained on one full day of real traffic scores it. Results go to PostgreSQL and a Grafana dashboard that refreshes itself every 5 seconds.

On a held-out day with 100 synthetic attacks mixed in, the system flags 87% of attacks at intensity 3 to 5 at a cost of 17.3 false alarms per hour, most of them in five hours when the normal mix of bot and human traffic shifted. Live, an injected attack appears on the dashboard within 7 to 23 seconds, and the pipeline ran for 2 hours unattended with flat memory and no lost or double-counted edits.

1. What Is Real-Time Streaming Analytics?

A batch system collects data first and analyses it later, like downloading a file and opening it in Excel. A streaming system analyses data as it arrives, so its results are always current. Every streaming project is built from the same six layers: data is produced, transported, processed, scored by a model, stored, and shown on a live view.

Wikipedia suits this well. English Wikipedia receives about 150 edits per minute, every edit is published on a free public stream within seconds, and the traffic contains real events worth detecting: vandalism waves, edit wars and runaway bots. Unlike a replayed CSV file, the data is live.

The most important idea in the project is that streaming ingestion does not make a dashboard live by itself. A dashboard is real-time only if it (a) reads fresh data and (b) refreshes on its own. Section 8 explains how WikiRadar meets both conditions.

2. What the Mini-Project Contains

The project implements all six required layers. Figure 1 shows how they connect and the table lists the tool chosen for each.

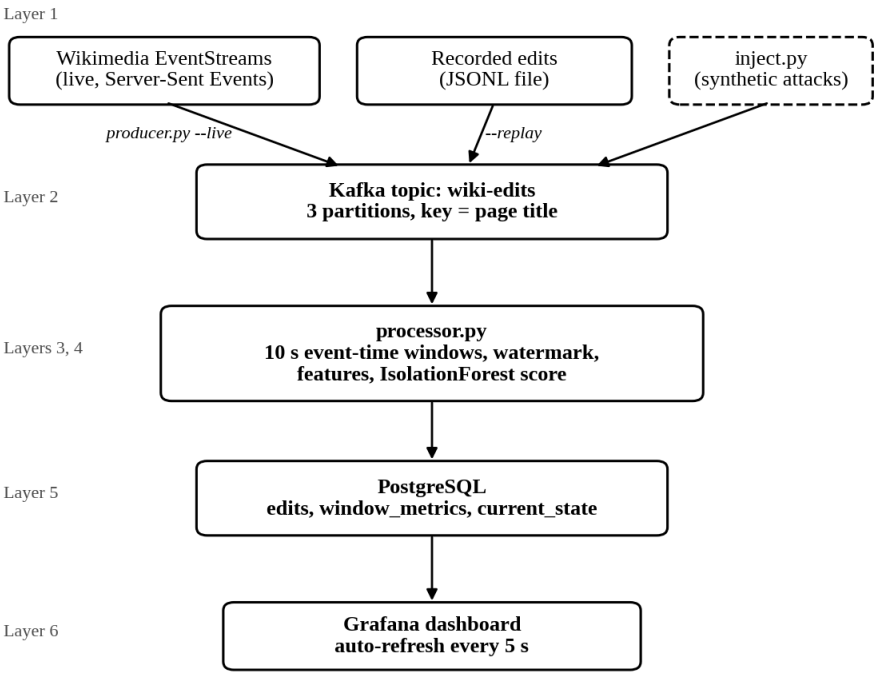


Figure 1: The six layers of WikiRadar. Data flows from top to bottom.

#	Layer	What it does	Tool chosen	Required
1	Data source	The event stream	Wikimedia EventStreams `recentchange` (live), recorded JSONL for replay	Any source
2	Streaming tool	Transports the stream	Apache Kafka 4.3 (KRaft mode)	Yes
3	Stream processor	Computes live metrics	Python, event-time tumbling windows with a watermark	Required
4	ML algorithm	Scores each window	scikit-learn IsolationForest	Yes
5	Database	Stores state and history	PostgreSQL 16	Any store
6	Visualization	Live auto-refresh view	Grafana 12	Yes

Deliverables: (1) working code for all layers in the WikiRadar repository, (2) this report, and (3) a screen recording of the dashboard updating as edits stream in. Kafka, PostgreSQL and Grafana run in Docker; the Python services run natively on an Apple Silicon MacBook.

3. The Dataset: Live Wikipedia Edits

The source is the Wikimedia `recentchange` stream (`https://stream.wikimedia.org/v2/stream/recentchange`), which carries every change on every Wikimedia wiki as Server-Sent Events. WikiRadar keeps only English Wikipedia (`wiki ==`

"enwiki") edits and page creations (type is edit or new): in a sample, 938 of 26,719 events (3.5%). One real event, with the username redacted:


```
{
  "meta": { "id": "4b39663b-9d48-4376-afd0-9ce1385c4893",
            "dt": "2026-10-03T10:34:23.164Z" },
  "type": "edit",
  "title": "Birlenbach",
  "title_url": "https://en.wikipedia.org/wiki/Birlenbach",
  "comment": "/* Transport links */ typo",
  "user": "[redacted]",
  "bot": false,
  "length": { "old": 1971, "new": 1961 },
  "revision": { "old": 1377573905, "new": 1378187223 },
  "wiki": "enwiki"
}
```


Each event is reduced to an Edit record: event id, event time dt, title and link, user, bot flag, anonymous flag (an IP address or a temporary account whose name starts with ~), new-page flag, change in size in bytes (here -10), revert flag (from the comment) and revision id. Events with missing fields are counted and dropped instead of crashing the producer.

The stream can replay roughly the last 7 days (?since=), so the training and test data were recorded in under an hour:

File	Period (UTC)	Edits	10 s windows
train_24h.jsonl	Oct 3 10:00 to Oct 4 10:00	226,343	8,640
holdout_24h.jsonl	Oct 4 10:00 to Oct 5 10:00	216,653	8,640

Both files cover the full 24 hours with no duplicates and no gap longer than about 11 seconds. A window holds about 26 edits on average. Traffic follows a strong daily cycle: on the training day it ranged from 4,643 to 13,226 edits per hour, and the bot share ranged from 7% to 53% by hour. About 6% of edits come from anonymous users.

4. Layers 1 and 2: Streaming the Data with Kafka

Kafka is a message broker: a durable pipe that carries events from a producer to a consumer and lets them run at different speeds. WikiRadar uses one broker in KRaft mode (no ZooKeeper) and one topic, wiki-edits, with 3 partitions. Docker Compose starts Kafka, PostgreSQL and Grafana, and a one-shot kafka-init container creates the topic.

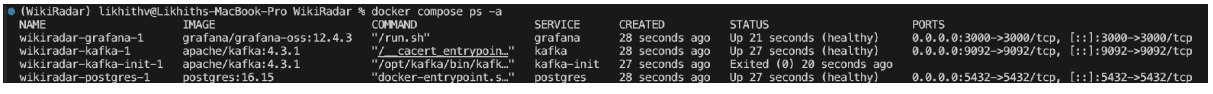


Figure 2: Docker services running and healthy (docker compose ps -a).

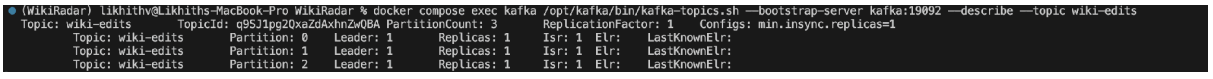


Figure 3: The Kafka topic wiki-edits with 3 partitions.

4.1 The producer

scripts/producer.py has three modes: --live streams from Wikimedia into Kafka, --record writes to a file (with --since for backfill), and --replay plays a file back into Kafka. The

connection is a long-lived HTTP stream. When it drops (11 times in a 2-hour run), the client reconnects and sends the id of the last event it saw, so Wikimedia resumes exactly where it left off:

```
headers = {"User-Agent": self.user_agent, "Accept": "text/event-stream"}
params = None
if self.last_id:
    headers["Last-Event-ID"] = self.last_id
elif self.since:
    params = {"since": self.since}
```

Wikimedia rejects clients without a descriptive User-Agent; after two refusals in a row (HTTP 403 or 429) the producer stops instead of retrying forever. Replay re-stamps event times to the present, so replayed and injected edits share one timeline, and must run at speed 1 so each window holds a normal number of edits.

4.2 The topic and the producer settings

Each message's key is the page title, so all edits to one page stay in order on one partition. The producer waits for the broker to confirm every write and uses Kafka's idempotent producer, so a network retry cannot write the same message twice:

```
Producer({
    "bootstrap.servers": config.KAFKA_BOOTSTRAP,
    "acks": "all",
    "linger.ms": 20,
    "enable.idempotence": True,
})
```

Kafka is what lets the processor be stopped and restarted without losing edits: messages stay in the topic, and the processor's consumer group remembers how far it has read.

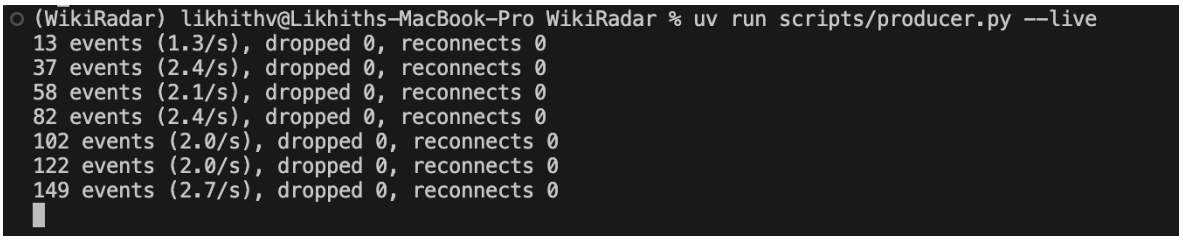


Figure 4: Producer in live mode, printing its event rate every 10 seconds.

5. Layer 3: The Stream Processor (Windowing)

The processor consumes each edit and keeps state in memory between events. Instead of a sliding window over the last N readings, WikiRadar uses 10-second **tumbling** windows (12:00:00 to 12:00:10, 12:00:10 to 12:00:20, and so on), because Wikipedia edits arrive at irregular times and the model needs fixed-length summaries.

5.1 Event time, not processing time

Windows are assigned by **event time**, the moment the edit was made on Wikipedia, not by **processing time**, the moment the processor reads it. After a restart the processor reads several minutes of backlog in a few seconds. With processing-time windows that backlog would collapse into one huge window; with event-time windows each edit lands in the window it belongs to.

5.2 The watermark

With event time, the processor never knows for sure that a window is complete, because an older edit could still arrive. The **watermark** decides: it is the latest event time seen minus 5 seconds (tracked per partition, as explained below), and once a window's end is behind it the window is closed, scored and written. An edit that arrives after its window has closed is counted as late and not added. In normal running no edits were late.

The first version used one watermark for the whole stream. A restart test showed that Kafka delivered one partition's backlog first, which pushed the watermark forward and caused 43 edits from the other partitions to be dropped as late. The fix, the same idea as Flink's per-partition watermarks, is a watermark per partition: a window closes only when the slowest partition has moved past it.

```
def add(self, edit: Edit, source: int = 0) -> list[ClosedWindow]:
    start = window_start(edit.dt)
    if self.closed_until and start < self.closed_until:
        self.late_dropped += 1
        return []
    self.buckets.setdefault(start, []).append(edit)
    self.waiting.discard(source)
    self.source_max[source] = max(self.source_max.get(source, edit.dt), edit.dt)
    return self._advance()

def _advance(self) -> list[ClosedWindow]:
    if self.waiting or not self.source_max:
        return []
    return self._close_until(min(self.source_max.values()) - LATENESS)
```

A partition that receives nothing for 3 seconds while the others keep delivering stops holding the watermark back, so a quiet partition cannot freeze the dashboard. Windows with no edits are still written with zero counts, because a sudden silence is information too. `WindowAggregator` has no input or output of its own, so training, live processing and evaluation all use the same code.

```
(WikiRadar) likhithv@Likhiths-MacBook-Pro WikiRadar % uv run scripts/processor.py
processor: 3 partitions, resuming after window ending 2026-10-12 13:40:20+00:00
processor: 3 partitions, resuming after window ending 2026-10-12 13:40:20+00:00
ANOMALY 13:40:30 score +0.057 top_title_share high (z=+8.7)
windows up to 13:40:40, late 0, dropped 0, skipped on resume 0
windows up to 13:40:50, late 0, dropped 0, skipped on resume 0
windows up to 13:41:00, late 0, dropped 0, skipped on resume 0
lag 10.7s (now minus latest closed window end), open windows 2
windows up to 13:41:10, late 0, dropped 0, skipped on resume 0
windows up to 13:41:20, late 0, dropped 0, skipped on resume 0
windows up to 13:41:30, late 0, dropped 0, skipped on resume 0
lag 11.2s (now minus latest closed window end), open windows 2
windows up to 13:41:40, late 0, dropped 0, skipped on resume 0
windows up to 13:41:50, late 0, dropped 0, skipped on resume 0
windows up to 13:42:00, late 0, dropped 0, skipped on resume 0
lag 11.7s (now minus latest closed window end), open windows 2
windows up to 13:42:10, late 0, dropped 0, skipped on resume 0
windows up to 13:42:20, late 0, dropped 0, skipped on resume 0
windows up to 13:42:30, late 0, dropped 0, skipped on resume 0
lag 12.1s (now minus latest closed window end), open windows 2
windows up to 13:42:40, late 0, dropped 0, skipped on resume 0
windows up to 13:42:50, late 0, dropped 0, skipped on resume 0
windows up to 13:43:00, late 0, dropped 0, skipped on resume 0
lag 12.5s (now minus latest closed window end), open windows 2
ANOMALY 13:43:00 score +0.026 anon_ratio high (z=+8.3) [synthetic]
windows up to 13:43:10, late 0, dropped 0, skipped on resume 0
windows up to 13:43:20, late 0, dropped 0, skipped on resume 0
windows up to 13:43:30, late 0, dropped 0, skipped on resume 0
```

Figure 5: Processor output: closed windows, lag, a real ANOMALY and the injected [synthetic] one.

6. Layer 4: The Machine-Learning Model

WikiRadar uses an IsolationForest from scikit-learn. It is unsupervised: it needs no labelled examples, which matters because nobody labels Wikipedia attacks in real time. It learns what normal traffic looks like and flags the rest. The model builds 200 random trees that split the data on random features at random values; unusual points are isolated after few splits, normal points need many.

6.1 The train/serve split

1. **Train offline (once).** `scripts/train.py` fits the model on the recorded training day and saves it to `models/iforest.joblib`.
2. **Serve live (always).** The processor loads the saved model once at startup and scores each closed window. The stream never retrains.

A common bug in such systems is training/serving skew: the live code computes a feature slightly differently from the training code. WikiRadar avoids it by construction. Windowing and features exist only in `wikiradar/features.py`, which training, the processor and the evaluation all import, and the model file stores its feature names so the processor refuses to start if they do not match.

6.2 Training the model

Each window becomes 11 features:

Feature	Meaning
`edit_count`	log(1 + number of edits)
`bot_ratio`, `anon_ratio`	Share of edits by bots, and by anonymous users
`new_page_count`	log(1 + new pages)
`mean_abs_delta`	log(1 + average size of a change in bytes)
`min_delta`	The most negative size change, on a signed log scale
`revert_ratio`	Share of edits whose comment says they undo another edit
`top_title_share`	Share of edits that went to the single most edited page
`unique_users_ratio`	Distinct users divided by edits
`hour_sin`, `hour_cos`	The UTC hour of the window as a point on a circle

```
def fit(X: np.ndarray, contamination: float = 0.02) -> dict:
    model = IsolationForest(n_estimators=200, contamination=contamination,
                            random_state=42).fit(X)
    return {"model": model, "feature_names": list(FEATURE_NAMES),
            "train_mean": X.mean(axis=0), "train_std": X.std(axis=0), ...}
```

The key setting is `contamination`, the share of training windows the model treats as anomalous, which sets the threshold. The initial value of 0.01 caught too few attacks, so 0.02 was chosen after the sweep in Section 11.2. It is a direct trade-off: a lower value raises fewer false alarms but misses weaker attacks, and a higher value catches more attacks at the cost of more false alarms. The hour features let the model learn the daily cycle, so the quiet early morning is not flagged every day; sine and cosine make 23:00 and 00:00 neighbours.

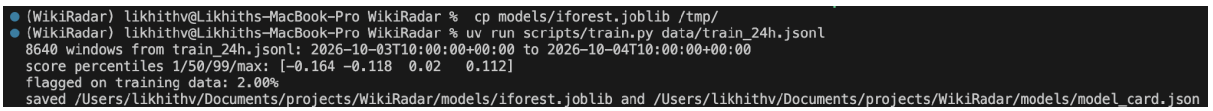


Figure 6: Training output: number of windows, score percentiles and share flagged.

6.3 Scoring live in the processor

scikit-learn's `decision_function` is positive for normal points and negative for outliers. WikiRadar flips its sign, so a higher score means more unusual and a score above 0 means flagged. Each flagged window also gets a plain-language reason: the feature furthest from its training average in standard deviations, for example `anon_ratio high (z=+8.6)`.

```
artifact = model.load()          # once, at startup

def score(artifact: dict, x: np.ndarray) -> tuple[float, bool, str]:
    s = float(scores(artifact, x.reshape(1, -1))[0])
    return s, s > 0, reason(x, artifact["train_mean"], artifact["train_std"])
```

7. Layer 5: The Database (Serving Layer)

The database stores what the dashboard reads. This decouples processing from visualization: the processor writes, the dashboard reads, and neither waits for the other. WikiRadar uses three tables:

- `edits`: every edit with its flags and size change, kept for 24 hours. Feeds the most-edited-pages table.
- `window_metrics`: one row per 10-second window with its features, score, anomaly flag and reason. Feeds the charts and the anomalies table.
- `current_state`: a single row, updated every window, holding edits per minute, bot and anonymous share, and anomalies in the last hour. Feeds the summary tiles. Current state is one thing, not a growing list, so one row keeps it small and fast.

Kafka guarantees **at-least-once** delivery: after a crash, messages since the last saved position are delivered again. The database makes those repeats harmless. Edits are keyed by their unique event id and duplicates are ignored; windows are keyed by their start time and overwritten:

```
INSERT INTO edits (...) VALUES (...) ON CONFLICT (event_id) DO NOTHING;
INSERT INTO window_metrics (...) VALUES (...)
    ON CONFLICT (window_start) DO UPDATE SET ...;
```

The processor saves its Kafka position only after the database write has committed, and only up to the first edit of any window that is still open, so a restart rebuilds unfinished windows exactly. A `kill -9` crash test and a 60-second Kafka freeze both ended with every window's count equal to the edits stored and no duplicates.

```
 edits | windows | anomalies
-------+---------+-----------
 12448 |     696 |        57
(1 row)

      window_start      | n_events | score  | is_anomaly |              reason
------------------------+----------+--------+------------+----------------------------------
 2026-10-12 15:16:20+00 |       20 | -0.072 | f          | revert_ratio high (z=+2.3)
 2026-10-12 15:16:10+00 |       16 | -0.074 | f          | unique_users_ratio high (z=+1.9)
 2026-10-12 15:16:00+00 |       32 |  0.067 | t          | top_title_share high (z=+19.4)
 2026-10-12 15:15:50+00 |       16 | -0.045 | f          | unique_users_ratio high (z=+2.3)
 2026-10-12 15:15:40+00 |       25 | -0.080 | f          | unique_users_ratio high (z=+1.8)
 2026-10-12 15:15:30+00 |       34 | -0.103 | f          | anon_ratio high (z=+1.5)
 2026-10-12 15:15:20+00 |       25 | -0.103 | f          | anon_ratio high (z=+1.7)
 2026-10-12 15:15:10+00 |       19 | -0.070 | f          | top_title_share high (z=+2.5)
 2026-10-12 15:15:00+00 |       39 |  0.050 | t          | anon_ratio high (z=+8.1)
 2026-10-12 15:14:50+00 |       11 | -0.035 | f          | unique_users_ratio high (z=+2.3)
(10 rows)
```

Figure 7: Row counts and the latest windows in PostgreSQL.

8. Layer 6: The Live Dashboard

A dashboard feels frozen for two independent reasons, and both must be fixed:

1. **No new data is arriving.** WikiRadar reads a live stream (or replays at real speed), and the processor writes a new window every 10 seconds.
2. **The view does not re-query.** Grafana re-runs every query every 5 seconds without any click.

The dashboard is defined in a JSON file that Grafana loads at startup, so a fresh installation shows it with no clicks and no login. Panels:

- Tiles for edits per minute, bot share and anonymous share over the last 5 minutes, and anomalies in the last hour.
- Edits per window and the anomaly score over time, both with red markers on flagged windows.
- A stacked chart of bot, human and anonymous edits per window.
- A table of recent anomalies with score, reason, most edited page and a synthetic flag.
- The most edited pages of the last 5 minutes, each linking to the real Wikipedia page.

Every query is limited to the dashboard's time range, for example:

```
SELECT window_start AS time, n_events AS edits
FROM window_metrics WHERE $__timeFilter(window_start) ORDER BY 1
```

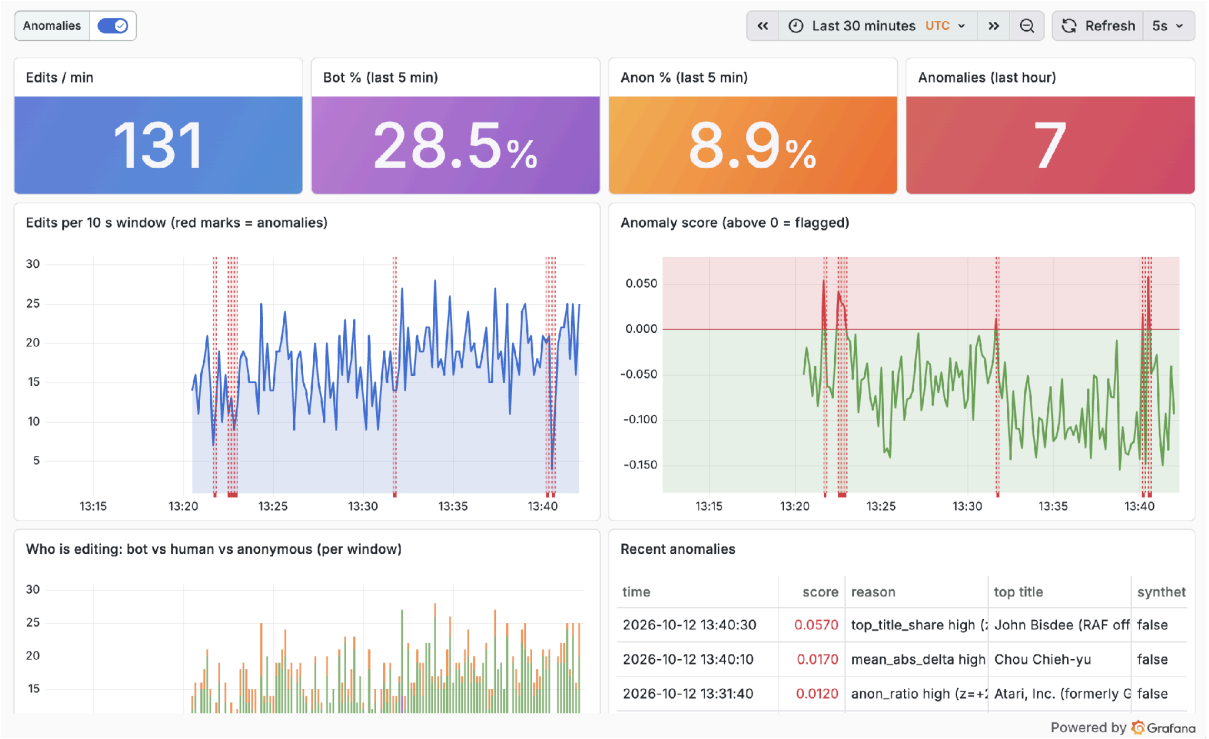


Figure 8: WikiRadar dashboard running on live Wikipedia traffic.

9. Running the Whole Pipeline

Each part runs in its own terminal, in this order:

1. Start Kafka, PostgreSQL and Grafana, and create the topic: `docker compose up -d` (the `kafka-init` container creates `wiki-edits`).
2. The trained model (`models/iforest.joblib`) is included in the repository, so no training step is needed. To retrain: `uv run scripts/train.py data/train_24h.jsonl`.
3. Start the stream processor (loads the model, writes to the database): `uv run scripts/processor.py`.
4. Open the dashboard: `http://localhost:3000/d/wiki-radar`.
5. Start the producer: `uv run scripts/producer.py --live`, or offline `uv run scripts/producer.py --replay data/sample_demo.jsonl --speed 1`.

To check that data is flowing, query the database. The window count should grow by about 6 per minute:

```
docker compose exec postgres sh -c 'psql -U "$POSTGRES_USER" -d "$POSTGRES_DB" \
  -c "SELECT count(*) FROM window_metrics"'
```

10. Demonstrating It Live

The demonstration follows this sequence:

- Reset the tables (`uv run scripts/reset_db.py`) and move the processor's Kafka position to the newest message, so counts start from zero.

- Start the processor, open the empty dashboard, then start the producer. The tiles count up and the curves draw themselves.
- Inject a synthetic attack into the same Kafka topic: `uv run scripts/inject.py --attack vandal_wave --intensity 3`. Typically within 20 seconds (at most about 25) a red marker appears and the anomalies table shows `anon_ratio high`.
- Inject an edit war: `uv run scripts/inject.py --attack edit_war --intensity 4`. It is flagged for a different reason, `top_title_share high`.
- Pause the producer (Ctrl-Z). The page keeps refreshing but the numbers freeze, proving that live depends on data moving, not just on the refresh timer. Resume with `fg`; the missed edits still land in the right windows because windows use event time.

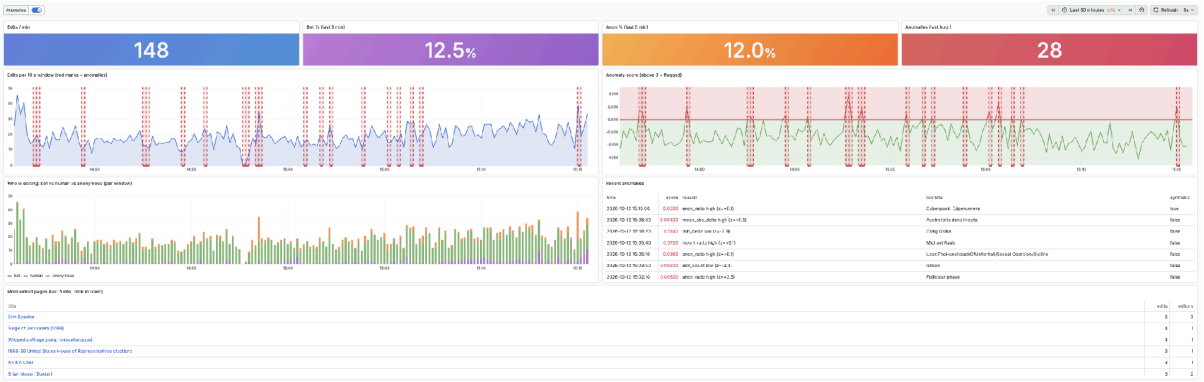


Figure 9: Dashboard after injecting a vandal_wave attack: red marker and a new row in Recent anomalies.

Recent anomalies

time	score	reason	top title	synthetic
2026-10-12 15:16:00	0.0670	top_title_share high (z=+19.4)	Synthetic page 287784	true
2026-10-12 15:15:00	0.0500	anon_ratio high (z=+8.1)	Cyberpunk: Edgerunners	true
2026-10-12 15:06:50	0.00400	mean_abs_delta high (z=+5.5)	Austroliotia densilineata	false
2026-10-12 15:06:20	0.0140	min_delta low (z=-2.9)	Craig Osika	false
2026-10-12 15:05:40	0.0720	revert_ratio high (z=+5.1)	Michael Raab	false
2026-10-12 15:05:10	0.0360	anon_ratio high (z=+6.1)	User:TheHunchbackOfUstlerhall/Sexual Coercion/Outline	false
2026-10-12 15:03:50	0.00600	edit_count low (z=-2.3)	Islikon	false

Figure 10: Edit war injection flagged with a different reason.

11. Results

11.1 Offline evaluation

The evaluation runs offline through the same windowing and feature code, which makes it fast (16 seconds) and repeatable. 100 synthetic attacks (5 types, intensities 1 to 5, each combination 4 times, fixed seed) were mixed into the holdout day:

Attack	What it simulates
`vandal_wave`	Many anonymous users removing text from many pages
`edit_war`	2 or 3 users reverting each other on one page
`bot_flood`	One bot account editing at a very high rate
`page_blank`	A few edits removing 20 kB to 100 kB each

Attack	What it simulates
`spam_pages`	A burst of new pages from new or anonymous accounts

Intensity scales each attack from 1 to 5: it multiplies the number of edits, and for `page_blank` also the size of each removal. Each attack is placed inside one 10-second window and counts as detected if that window is flagged. The "beyond chance" figure counts only windows that the attack itself pushed over the threshold.

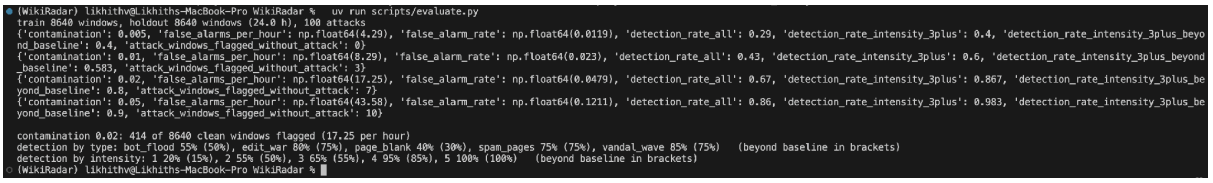


Figure 11: Output of the offline evaluation script.

11.2 Contamination sweep

Contamination	False alarms per hour	Detected, intensity 3 to 5	Detected, all
0.005	4.3	40%	29%
0.01	8.3	60%	43%
0.02 (chosen)	**17.3** (4.8% of windows)	**87%** (80% beyond chance)	**67%**
0.05	43.6	98%	86%

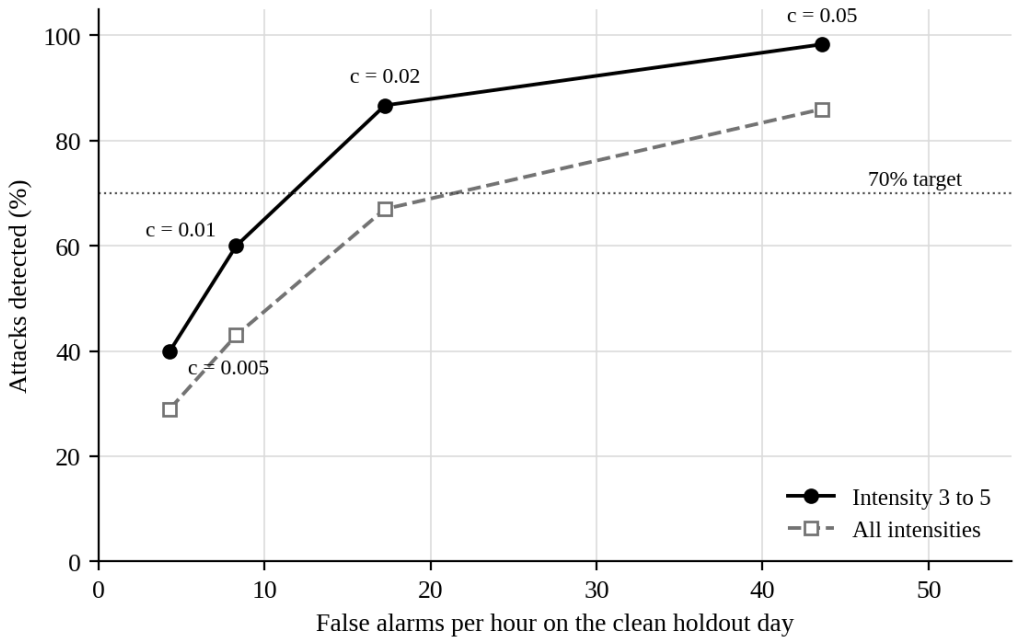


Figure 12: Contamination sweep: detection rate against false alarms per hour.

The target was 70% detection at intensity 3 and above. The initial contamination of 0.01 reached only 60%, so 0.02 was chosen: it doubles the false alarms but clears the target even when chance detections

are removed. The value was chosen on the same holdout attacks it is measured on, so 87% is somewhat optimistic.

11.3 Detection by attack type and intensity

Attack type	Detected	Intensity	Detected
vandal_wave	85%	1	20%
edit_war	80%	2	55%
spam_pages	75%	3	65%
bot_flood	55%	4	95%
page_blank	40%	5	100%

Edit wars are easy because one page suddenly receives a large share of all edits. Bot floods are hard because about a third of normal traffic already comes from bots. IsolationForest is also weak on anomalies that are extreme in only one feature, because each tree splits on randomly chosen features; attacks that move several features at once, like a vandal wave, are caught far more reliably.

False alarms. 356 of the 414 false alarms fall between 05:00 and 10:00 UTC on Oct 5; the other 19 hours average 3.1 per hour, under 1% of windows. In those five hours traffic dropped to 4,400 to 5,500 edits per hour with only 14% to 19% bots. On the training day the same quiet, low-bot pattern happened at 10:00 UTC, so a model that had seen each hour only once found it strange at 06:00.

Real-world check. In the 20 highest-scoring real windows of the holdout day, 7.0% of edits (11 of 158) were later tagged by Wikipedia as reverted, against 3.1% (14 of 445) in 20 random normal windows. The samples are small, so this is a hint, not proof.

11.4 Live behaviour and reliability

Test	Result
Training	8,640 windows, 2.00% flagged by design
Injection to red marker	16 to 23 s when the attack starts at the beginning of a window; 7 to 10 s when it lands later in a window
2 hours unattended, live	No crash; 11 stream disconnects all resumed; processor lag mean 12.7 s, max 19.6 s; memory flat (processor about 180 MB, producer about 46 MB)
Data integrity after that run	731 windows, no gaps, no duplicate edits, every window's count equal to the edits stored
Crash and stall	`kill -9` restart and a 60 s Kafka freeze: processor recovers, every window still matches the stored edits
Fresh install	A copy of only the repository files, following the README, filled the dashboard within 75 s
Demonstration run (Oct 12, about 2 h, live)	12,448 edits in 696 windows; 57 windows flagged (8.2%); all 3 injected attacks flagged in the window they landed in; 0 late or dropped edits; producer 2.0 to 2.7 edits/s; processor lag 10.7 to 14.8 s

The detection delay has a floor: a window is 10 seconds long and closes 5 seconds after its end, and the browser adds up to 5 seconds until its next refresh. In the demonstration run, a vandal wave injected at 13:43:03 landed in the 13:43:00 window, which closed at 13:43:10 and was flagged about 5 seconds later (Figure 5).

The demonstration run flagged 8.2% of windows, more than the 4.8% measured on the holdout day. The real (non-injected) flags visible in Figures 9 and 10 each have a single feature moderately above normal, for example `mean_abs_delta high (z=+5.5)` or `revert_ratio high (z=+5.1)`, while the injected attacks score much higher on their main feature (`anon_ratio` z=+8.1, `top_title_share` z=+19.4). This matches the holdout finding: traffic that drifts away from the one training day raises the false-alarm rate. Training on more days, or a stricter threshold for the live view, would reduce it.

11.5 Automated tests

A pytest suite covers event parsing (anonymous and temporary accounts, revert detection, missing fields), window alignment, the watermark and late events, out-of-order events, feature values for a hand-built window, the attack generators and the model's scoring and reason text.

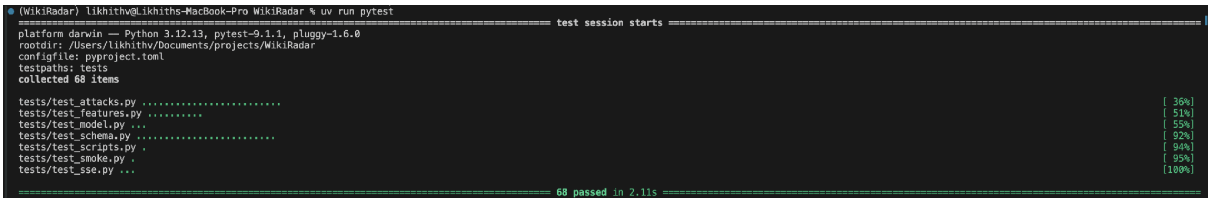


Figure 13: Test suite passing.

12. Limitations and Future Work

- **No labels.** Detection was measured only on synthetic attacks designed for this project; real vandalism is messier. Synthetic attacks are a proxy: the detection rates say how well the model separates these five patterns from normal traffic, not how many real incidents it would catch.
- **One training day.** The model saw each hour once, which caused the cluster of false alarms on Oct 5. A week of training data would fix this.
- **Threshold chosen on the test data.** A separate validation day would give an unbiased detection rate.
- **Single-node Kafka.** No replication; fine for a demo, not for production.
- **Window granularity.** The 10-second window and 5-second watermark set a 15-second floor on detection delay, and intensity-1 attacks are mostly invisible among about 26 normal edits.

Future work: train on a full week, add an ARIMA forecast of edits per minute plotted ahead of the live curve, add a per-edit model on top of the window model, and run the processor on Apache Flink to compare it with the Python implementation.

13. Conclusion

WikiRadar is a complete streaming pipeline from a public live source to a self-updating dashboard. Kafka buffers and replays the feed, the processor windows edits by event time using a per-partition

watermark, an IsolationForest trained on a day of real traffic scores every window, and PostgreSQL and Grafana present the results typically within 15 to 20 seconds.

The design choices that mattered most were event-time windows, idempotent writes with offsets saved only after the database commit, and keeping all feature code in one shared module. They are why a crash, a broker stall or a replay leaves every window correct, and why the offline evaluation measures exactly what runs live. The evaluation also shows the method's limits: the model finds strong, multi-signal attacks reliably but struggles with attacks that hide in normal bot traffic, and it mistakes an unusual but harmless shift in daily traffic for an anomaly.

References

[1] Wikimedia Foundation. EventStreams. https://wikitech.wikimedia.org/wiki/Event_Platform/EventStreams

[2] F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation Forest," in Proc. IEEE International Conference on Data Mining (ICDM), 2008, pp. 413-422.

[3] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[4] Apache Software Foundation. Apache Kafka Documentation. https://kafka.apache.org/documentation/

[5] T. Akidau et al., "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing," Proc. VLDB Endowment, vol. 8, no. 12, 2015.

[6] Grafana Labs. Grafana Documentation: Provisioning. https://grafana.com/docs/grafana/latest/administration/provisioning/

Appendix: Submission Checklist

Requirement	**How WikiRadar meets it**	**Status**
At least one streaming tool, working	Apache Kafka 4.3, topic `wiki-edits` (Section 4)	Done
At least one ML algorithm, trained and scoring live	IsolationForest, trained offline, scores every window live (Section 6)	Done
At least one visualization tool, auto-refreshing on a live source	Grafana, 5 s refresh on PostgreSQL (Section 8)	Done
A database serving layer between processor and dashboard	PostgreSQL 16 with `edits`, `window_metrics`, `current_state` (Section 7)	Done
Full pipeline runs end to end, dashboard updates on its own	Section 9; 2-hour unattended live run (Section 11.4)	Done
Written report and live demo or screen recording	This report; screen recording submitted separately	Done